

CRD INTRO BY AKSHAT (AKSHATAWSAC@GMAIL.COM)

CRD = Custom Resource Definition

Normally Kubernetes understands only built-in resources:

```
kind: Pod
kind: Service
kind: Deployment
kind: ConfigMap
```

Suppose your company wants Kubernetes to understand a new resource called:

```
<> YAML
```

```
kind: Database
```

or

```
<> YAML
```

```
kind: RedisCache
```

or

```
<> YAML
```

```
kind: Employee
```

Kubernetes doesn't know these resources.

So we create a **CRD**.

A CRD tells Kubernetes:

"From today onwards, a new resource called RedisCache exists."

After creating a CRD:

kubectl get redisCache

starts working

FLIPKART USE CASE

Without CRD in Flipkart

Imagine Flipkart's DevOps team receives a request:

```
Team: Electronics Team
```

```
Requirement:
```

```
Need Redis Cluster
```

```
3 replicas
```

```
20 GB storage
```

```
Automatic backup
```

Without CRD, DevOps engineers manually create:

```
StatefulSet
```

```
Service
```

```
ConfigMap
```

```
PVC
```

```
Secrets
```

```
NetworkPolicy
```

Maybe 500-1000 lines of YAML.

Every team sends the same request again and again.

```
Mobile Team
```

```
Fashion Team
```

```
Electronics Team
```

```
Grocery Team
```

DevOps team gets frustrated because they are repeating the same work.

Flipkart DevOps Team Creates a CRD

They create:

kind: RedisCluster

Now Kubernetes understands a new resource.

What Application Team Does

Instead of opening a ticket, the application team creates:

```
apiVersion: flipkart.com/v1
kind: RedisCluster

metadata:
  name: cart-cache

spec:
  replicas: 3
  storage: 20Gi
```

kubectl apply -f redis.yaml

What Happens Behind the Scenes:

A Redis Operator is watching.

RedisCluster Created



Operator Detects



Creates StatefulSet



Creates Service



Creates PVC



Creates Secret



Redis Cluster Ready

Application team never touches:

Deployment
PVC
Service
ConfigMap

Why Flipkart Created CRD?

Problem 1: Repeated Work

Without CRD:

DevOps Team



Create Redis

Create Redis

Create Redis

Create Redis

Hundred of times

Problem 2: Human Errors

One engineer creates:

3 Replicas

Another creates:

2 Replicas

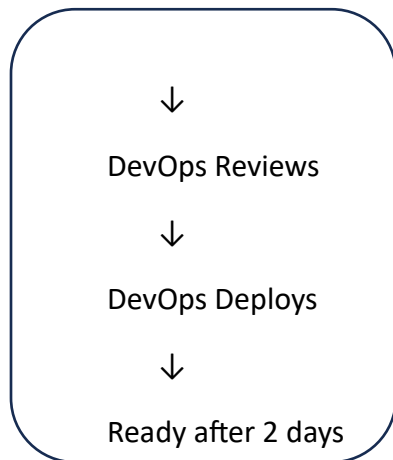
Another forgets backup configuration.

Every Redis cluster becomes different.

Problem 3: Slow Delivery

Application team waits:

Ticket Raised



After CRD

Application team simply writes:

```
kind: RedisCluster
```

and Kubernetes handles everything.

Ready in minutes.

Another Flipkart Example

Suppose Flipkart wants every microservice to have:



Instead of creating 5 YAMLs every time, DevOps creates:

```
kind: FlipkartApplication
```

Application team writes:

```
apiVersion: flipkart.com/v1
kind: FlipkartApplication
metadata:
  name: payment-service
spec:
```

replicas: 3
image: payment:v1

Operator automatically creates:



In companies like Flipkart, DevOps teams create CRDs to standardize and automate complex infrastructure deployments such as Redis, Kafka, PostgreSQL, or internal applications. CRDs allow application teams to deploy these services using simple custom resources while operators automatically create and manage the underlying Kubernetes resources.

CRD vs CR (custom resources)

CRD

Blueprint

kind: CustomResourceDefinition

name: rediscaches.cache.com

Defines:

RedisCache resource exists

CR

Actual object

apiVersion: cache.com/v1

kind: RedisCache

metadata:

name: myredis

This is a Custom Resource.

CRD Alone Does Nothing

Creating:

kind: RedisCache

does NOT create any pod.

Why?

Because Kubernetes only stores the object.

Some controller/operator must watch it.

CRD + Controller = Magic

User creates RedisCache

↓

Controller watches it

↓

Controller creates:

Pod

Service

PVC

Without Controller:

CRD = Just Data